

Revisão para a Prova

Processos de Software

Prova: 11/06 (qui) · Multiprova · 25 questões fechadas · 40 min

Consulta: 1 folha A4 manuscrita (frente e verso)

Como cai a prova

- **Escopo:** Unidades 1 e 2
- **Entra:** modelos de ciclo de vida, Manifesto Ágil, Scrum, XP, Lean, Kanban, DevOps (CI/CD, Docker), métricas DORA
- **NÃO entra:** VSM e controle de qualidade (*cobrados no projeto e na Proposta de Melhoria*)
- 40 min para 25 questões $\approx$ **1,5 min por questão**

1. Modelos de ciclo de vida

- **Cascata:** fases sequenciais (requisitos → projeto → implementação → testes → manutenção); pouco retorno
- **Modelo V:** cada fase de desenvolvimento tem uma fase de **verificação/teste** correspondente
- **Espiral:** iterativo e **orientado a riscos**, com prototipação
- **Iterativo/incremental:** entrega valor aos poucos, com feedback contínuo

2. Cascata vs Iterativo

- **Cascata:** escopo definido cedo; risco aparece **tarde** (na integração/teste)
- **Iterativo/incremental:** reduz risco entregando e validando ao longo do tempo
- Pegadinha: iterativo **não** elimina requisitos nem testes — apenas distribui no tempo

3. Manifesto Ágil — os 4 valores

Valorizamos os itens à **esquerda** mais que os à direita (que ainda têm valor):

1. **Indivíduos e interações** > processos e ferramentas
2. **Software funcionando** > documentação abrangente
3. **Colaboração com o cliente** > negociação de contratos
4. **Responder a mudanças** > seguir um plano

4. Scrum — os 3 papéis (accountabilities)

- **Product Owner:** maximiza o valor do produto; ordena o **Product Backlog**
- **Scrum Master:** serve ao time, remove impedimentos, garante o framework
- **Developers:** constroem o incremento; conduzem a Daily

Não existem "gerente de projeto" ou "testador" como papéis do Scrum.

5. Scrum — eventos

- **Sprint:** ciclo timeboxed (≤ 1 mês)
- **Sprint Planning:** define a meta e o Sprint Backlog
- **Daily Scrum:** diário, ≤ 15 min, dos Developers
- **Sprint Review:** inspeciona o **incremento** com stakeholders
- **Retrospective:** inspeciona o **processo** e planeja melhorias

6. Scrum — artefatos e DoD

- **Product Backlog** → o que pode ser feito (ordenado pelo PO)
- **Sprint Backlog** → trabalho selecionado + plano da Sprint
- **Incremento** → soma de itens "prontos"
- **Definition of Done (DoD)**: entendimento compartilhado do que torna o incremento pronto (qualidade/completude)

7. XP — valores e práticas

- **Valores:** comunicação, simplicidade, feedback, coragem, respeito
- **Práticas:** programação em par, **TDD**, integração contínua, refatoração, pequenas entregas, planning game
- **Pair programming:** revisão em tempo real → qualidade + disseminação de conhecimento

8. XP — TDD (red-green-refactor)

1. **Red:** escreve um teste que **falha**
2. **Green:** faz passar com o **mínimo** de código
3. **Refactor:** melhora o design mantendo os testes verdes

*Testes vêm **antes** do código de produção.*

9. Lean — princípios

- **Eliminar desperdício**
- Amplificar o aprendizado
- **Decidir o mais tarde possível** (postergar comprometimento irreversível)
- Entregar o mais rápido possível
- Empoderar o time
- Construir integridade
- Ver o todo

10. Lean — desperdícios (waste)

- Funcionalidades **não usadas**, trabalho **parcialmente** feito, trocas de contexto, esperas, retrabalho
- **Não** é desperdício: entrega de valor, aprendizado, testes, feedback
- "Decidir tarde" $\neq$ não decidir — é manter **opções** até ter informação

11. Kanban — fluxo e WIP

- Práticas centrais: **visualizar o fluxo** e **limitar o WIP** (work in progress)
- Limitar WIP → reduz multitarefa, **expõe gargalos**, melhora o tempo de entrega
- Kanban **não** exige sprints fixas nem papéis fixos

12. Kanban — métricas e Lei de Little

- **Lead time:** do pedido à entrega · **Cycle time:** do início ao fim do trabalho ·
Throughput: itens por período

- **Lei de Little (sistema estável):**

$$WIP = Throughput \times LeadTime$$

- Logo: menos WIP → menor lead time (com throughput estável)

13. DevOps — CI vs CD

- **Integração Contínua (CI):** integra e **valida** (build + testes) com frequência
- **Entrega Contínua (CD):** mantém o software **sempre pronto para liberar**
- **Implantação Contínua:** vai além — libera automaticamente em produção

14. Docker — imagem vs container

- **Imagem:** artefato **imutável** e versionável (a "receita")
- **Container:** instância em execução da imagem
- Containers **compartilham o kernel** do host ($\neq$ VM, que tem SO convidado completo)

15. DORA — as 4 métricas-chave

- **Frequência de deploy**
- **Lead time** para mudanças
- **Taxa de falha** em mudanças
- **Tempo para restaurar** serviço (MTTR)

→ Duas de **velocidade** (throughput) + duas de **estabilidade**

16. DORA — interpretação

- As quatro métricas devem ser lidas **em conjunto**
- Alta velocidade + instabilidade alta $\neq$ time "Elite"
- Arquétipos/clusters (Elite $\rightarrow$ Low) equilibram throughput e estabilidade
- Pegadinha: velocidade alta **não** torna estabilidade irrelevante

17. Armadilhas frequentes

- Inverter a leitura do Manifesto (itens da direita "não valem" → falso)
- Achar que Scrum tem "gerente de projeto"
- Confundir **Sprint Review** (produto) com **Retrospective** (processo)
- Misturar **CI** e **CD**
- Confundir **imagem** e **container**
- Esquecer a forma da Lei de Little

18. Antes da prova

- Monte sua **folha A4** com: 4 valores ágeis, papéis/eventos/artefatos do Scrum, princípios Lean, fórmula de Little, 4 métricas DORA
- Releia os **quizzes** e os slides das sprints
- Gerencie o tempo: ~1,5 min/questão; marque e volte nas difíceis

Boa prova! 🚀